



SWE 4603

Software Testing and Quality Assurance

Week 9

Prepared By Maliha Noushin Raida, Lecturer, CSE
Islamic University of Technology

Lesson Outcome

- Entities to be measured for the software: process, product, and resource
- Recognition of attributes before measurement, as they are important for designing software metrics
- Line-of-code metrics
- Halstead metrics
- Function point analysis metrics

Week on:

- Chapter 10: Software Metrics

Software Metrics

Measure

- ❖ provides a quantitative indication of the extent, amount, dimension, capacity or size of some attribute of a process or product.

Example: Number of defects found in component testing. LOC of each component.

Measurement

- ❖ The act of collecting a measure.
- ❖ Example: Collecting the defect counts. Counting LOC

Software Metrics

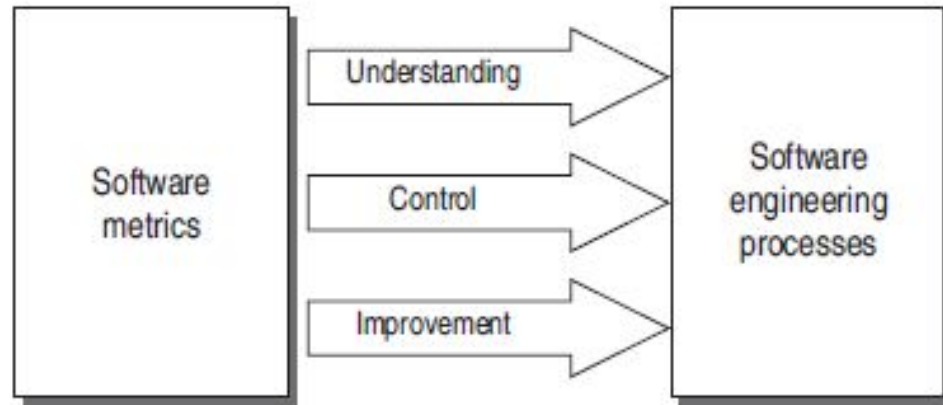
SW Metrics refers to a range of measurements for computer software that enable software people to gain insight into the project:

- ❖ To improve the Process and the Product
- ❖ Assist in Estimation
- ❖ Productivity Assessment
- ❖ Quality Control
- ❖ Project Control

Software Metrics

Need of SW Measurement:

- ❖ Measurements are a key element for controlling software engineering processes.
- ❖ software measurement is needed for the following activities:
 - Understanding
 - Control
 - Improvement



Software Metrics

Classification of SW Metrics

Product vs Process Metrics

- ❖ **Product metrics** are measures of the software product at any stage of its development, from requirements to installed system.
- ❖ **Product metrics** may measure the complexity of the software design, the size of the final program, or the number of pages of documentation produced.
- ❖ **Process metrics**, on the other hand, are measures of the software development process, such as the overall development time, type of methodology used, or the average level of experience of the programming staff.

Software Metrics

Classification of SW Metrics

Primitive vs computed Metrics

- ❖ Primitive metrics are those metrics that can be directly observed, such as the program size in LOC, the number of defects observed in unit testing, or the total development time for the project.
- ❖ Computed metrics are those that cannot be directly observed but are computed in some way from other metrics. For example, productivity metrics like LOC produced per person-month or product quality like defects per thousand lines of code.

Software Metrics

Classification of SW Metrics

Private vs Public Metrics

- ❖ This classification is based on the use of different types to process data.
- ❖ Examples of private metrics include defect rates (by individual and by module) and errors found during development.
- ❖ Public metrics integrate information that originally was private to individuals and teams. Project-level defect rates, effort, calendar times, and related data are collected and evaluated in an attempt to uncover indicators that can improve organizational process performance.

Software Metrics

Entities to be measured:

The entities considered in software measurement are:

Processes Any activity related to software development.

Product Any artifact produced during software development.

Resource People, hardware, or software needed for a process.

The attributes of an entity can be **internal** or **external**.

- ❖ SW size is an internal attribute of any software measurement.
- ❖ Productivity, an external attribute of a person, clearly depends on many factors such as the kind of process and the quality of the software delivered.

Software Metrics

Size Metric:

- ❖ Software size is an important metric to be used for various purposes.
- ❖ Difficult to Measure
- ❖ Various approaches are used to measure SW size
 - LINE OF CODE (LOC)
 - TOKEN COUNT (HALSTEAD PRODUCT METRICS)
 - FUNCTION POINT ANALYSIS (FPA)

Software Metrics

LINE OF CODE (LOC)

- ❖ This metric is based on the number of lines of code present in the program.
- ❖ The lines of code are counted to measure the size of a program.
- ❖ The **comments** and **blank** lines are ignored during this measurement.
- ❖ The LOC metric is often presented on thousands of lines of code (KLOC).

Software Metrics

TOKEN COUNT (HALSTEAD SOFTWARE METRICS)

- ❖ LOC is not consistent, because all lines of code are not at the same level.
- ❖ Some lines are more difficult to code than others.
- ❖ **Halstead** stated that any software program could be measured by counting the number of operators and operands.
- ❖ From these set of operators and operands, he defined a number of formulae to calculate the **vocabulary**, the **length**, and the **volume** of the software program.

Software Metrics

TOKEN COUNT (HALSTEAD SOFTWARE METRICS)

❖ Halstead Metric

Program Vocabulary

- ❖ It is the number of unique operators plus the number of unique operands as given below:

$$n = n1 + n2$$

where n = program vocabulary

$n1$ = number of unique operators

$n2$ = number of unique operands

Software Metrics

TOKEN COUNT (HALSTEAD SOFTWARE METRICS)

❖ Halstead Metric

Program Length

- ❖ It is the total usage of all the operators and operands appearing in the implementation.

$$N = N_1 + N_2$$

where N = program length

N₁ = all operators appearing in the implementation

N₂ = all operands appearing in the implementation

Software Metrics

TOKEN COUNT (HALSTEAD SOFTWARE METRICS)

❖ Few Halstead Metric

Program Volume

- ❖ The volume refers to the size of the program and it is defined as the program length times the logarithmic base 2 of the program vocabulary.

$$V = N \log_2 n$$

where V = program volume

N = program length

n = program vocabulary

Software Metrics

TOKEN COUNT (HALSTEAD SOFTWARE METRICS)

❖ Few Halstead Metrics

Program Difficulty

- ❖ The difficulty refers to the estimated difficulty of understanding or writing the program.

$$D = (n1 / 2) \times (N2 / n2)$$

where $n1$ = Number of distinct operators

$N2$ = Total number of operand occurrences

$n2$ = Number of distinct operands

Software Metrics

TOKEN COUNT (HALSTEAD SOFTWARE METRICS)

- ❖ **Few Halstead Metric**

- Effort**

- ❖ The effort requires developing or understanding of the code

$$E = V * D$$

where V = program volume

D = program difficulty

Software Metrics

Guideline for calculating operands and operators:

- ❖ All the variables and constants are considered as operands.
- ❖ Local variables with the same name, if occurring in different functions, are counted as unique operands.
- ❖ Function calls are considered as operators.
- ❖ The looping statements do ... while, while, and for are operators. The statements if, if ... else, are operators. The switch ... case statements are considered as operators.
- ❖ The reserve words, returns, default, continue, break, and sizeof are all operators.
- ❖ The brackets, commas, and semicolons are operators.
- ❖ The unary and binary operators are considered as operators. The & is considered an operator.
- ❖ In arrays, the array name and index are considered as operands, and [] is considered as an operator.
- ❖ Comments are not considered.

Software Metrics

Example:

```
Void swap (int a[ ], int i)
```

```
{
```

```
int temp;
```

```
Temp = a[i];
```

```
a[i] = a[i+1];
```

```
a[i+1] = temp;
```

```
}
```

Operands	Occurrences	Operators	Occurrences
swap	1	()	1
a	5	{ }	1
i	5	void	1
temp	3	int	3
1	2	[]	5
		,	1
		;	4
		=	3
		+	2
$n_1 = 5$	$N_1 = 16$	$n_2 = 9$	$N_2 = 21$

Software Metrics

FUNCTION POINT ANALYSIS (FPA)

- ❖ It is based on the idea that the software size should be measured according to the functionalities specified by the user.
- ❖ Therefore, FPA is a standardized methodology for measuring various functions of software from the user's point of view early in the development process.
- ❖ The size of an application is measured in **function points**.
- ❖ FP can be used for:
 - Estimate the effort or cost of code and test the software system
 - Forecast the number of components and/or the number of source lines in the projected system.

Software Metrics

FUNCTION POINT ANALYSIS (FPA)

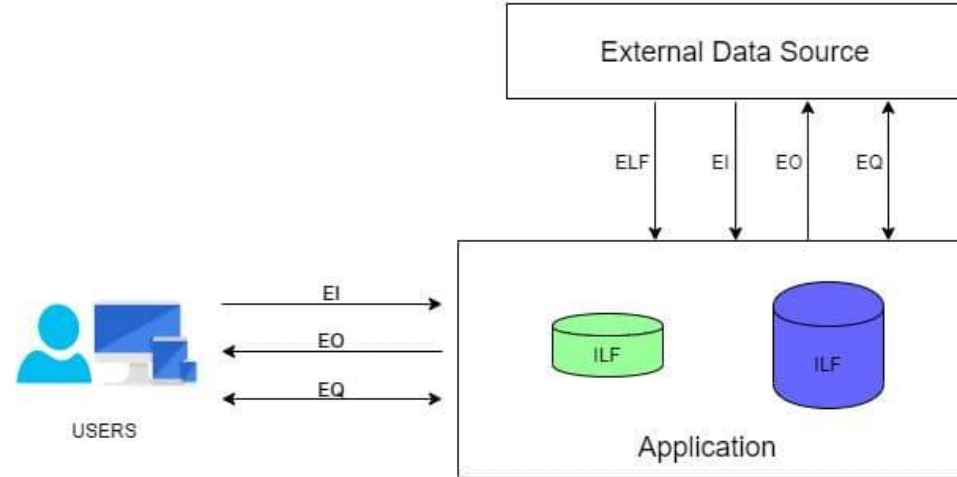
Process used to calculate the function points

1. Determine the type of project for which the function point count is to be calculated. For example, development project (a new project) or enhancement project.
2. Identify the counting scope and the **application boundary**.
3. Identify **data functions** (internal logical functions and external interface files) and their **complexity**.
4. Identify **transactional functions** (external inputs, external outputs, and external queries) and their **complexity**.
5. Determine the **unadjusted function point count (UFP)**.
6. Determine the **value adjustment factor (VAF)**, which is based on **14 general system characteristics (GSCs)**.
7. Calculate the **adjusted function point count (AFP)**.

Software Metrics

FUNCTION POINT ANALYSIS (FPA)

- ❖ The first step in calculating FP is to identify the counting boundary.
- ❖ Counting boundary: The border between the application or project being measured and external applications or the user domain.
- ❖ A boundary establishes which functions are included in the function point count

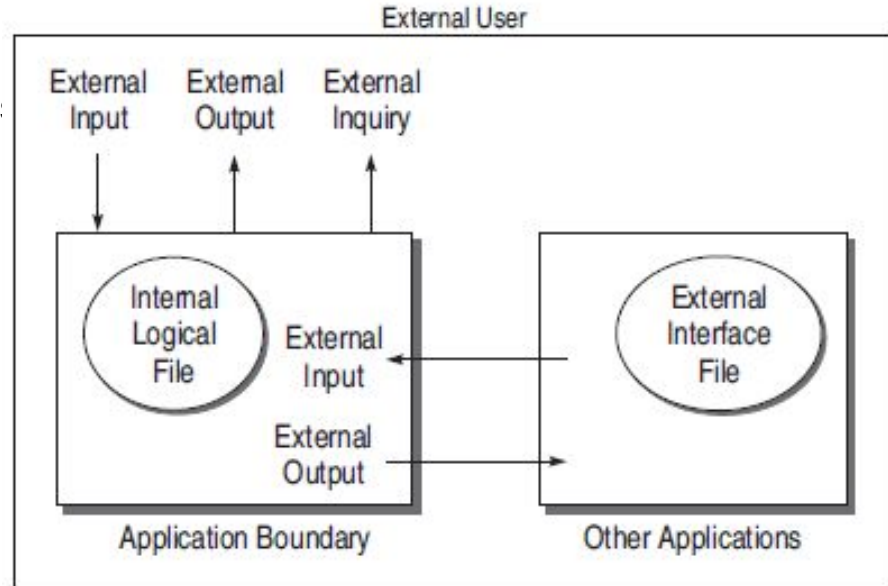


Software Metrics

FUNCTION POINT ANALYSIS (FPA)

❖ Two data function types:

- **Internal Logical Files (ILF):** these files are the master or transaction files that the system interacts with during its session.
- **External Interface Files (EIF):** represent the data that your application will use/reference, but data that is not maintained by your application.

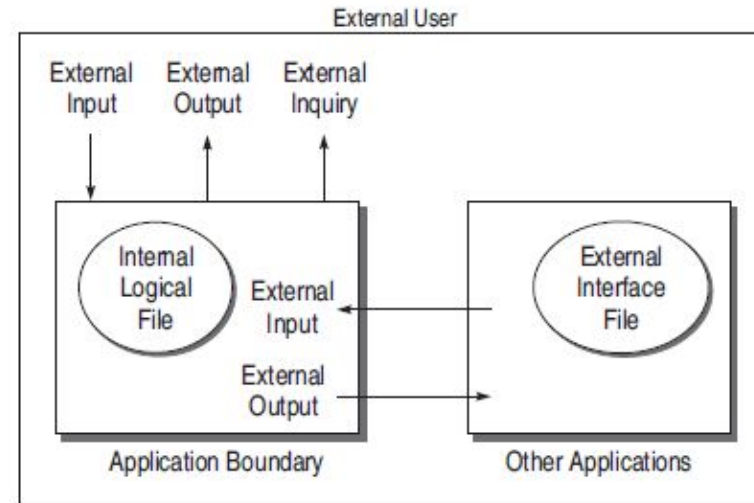


Software Metrics

FUNCTION POINT ANALYSIS (FPA)

❖ Three Transactional function types:

- **External Inputs (EI):** these are end-user actions such as putting in a login or executing a mouse click.
- **External Outputs (EO):** the system provides the end-user output or interface such as a GUI display or items in a report.
- **External Inquiries (EQ):** An External Inquiry is a transaction that allows the user to retrieve information from the system without updating it and without any significant processing. For example, the end-user wishes to submit a query to a database or requests on-line help.



Software Metrics

FUNCTION POINT ANALYSIS (FPA)

❖ Calculating UFP:

➤ Sizing Data Functions(ILF & EIF):

- The physical count of ILFs and EIFs, together with the relative functional complexity of each
- Each identified data function must be assigned a functional complexity based on the number of **data element types** (DETs) and **record element types** (RETs)
- DETs are unique user-recognizable, non-repeatable fields or attributes.
- RETs are user-recognizable subgroups (optional or mandatory) of data elements contained within an ILF or EIF.
- Functional complexity is the rating assigned to each data function by:

RETs	DETs		
	1-19	20-50	>=51
1	Low	Low	Average
2-5	Low	Average	High
>5	Average	High	High

Software Metrics

FUNCTION POINT ANALYSIS (FPA)

❖ Calculating UFP:

➤ Sizing Transactional Functions(EI, EO, and EQ):

- The physical count of EI, EOs, and EQ, together with the relative functional complexity of each
- Each identified data function must be assigned a functional complexity based on the number of **data element types** (DETs) and **file type referenced** (FTRs)
- An FTR must be either an Internal Logical File and/or External Interface File referenced by the EI/EO/EQ
- Complexity is the rating assigned to each transactional function by:

FTRs	DETs		
	1–4	5–15	>=16
<2	Low	Low	Average
2	Low	Average	High
>2	Average	High	High

For EI

FTRs	DETs		
	1–5	6–19	>=20
<2	Low	Low	Average
2–3	Low	Average	High
>3	Average	High	High

For EO

FTRs	DETs		
	1–5	6–19	>=20
<2	Low	Low	Average
2–3	Low	Average	High
>3	Average	High	High

For EQ

Software Metrics

FUNCTION POINT ANALYSIS (FPA)

❖ Calculating UFP:

- Count all five components, i.e. ILF, EIF, EI, EO, and EQ of an application and determine the level of the component as low, average, or high, using a set of prescriptive standards.
- Nevertheless, the determination of complexity is somewhat subjective.
- **Count the number of all five components. The sum of each count is multiplied by an appropriate weight using Table below.**

Components	Function Levels		
	Low	Average	High
ILF	X7	X10	X15
EIF	X5	X7	X10
EI	X3	X4	X6
EO	X4	X5	X7
EQ	X3	X4	X6

- **Add all five results calculated in the previous step. This is the final UFP.**

Software Metrics

❖ Calculating Adjusted Function Point:

- A **value adjustment factor (VAF)** is used as a multiplier of the unadjusted function point (UFP) count in order to calculate the **adjusted function point (AFP)** count of an application.
- To calculate the VAF, we evaluate the 14 GSCs(General system Characteristics) on a scale of 0–5 to determine the degree of influence(DI) for each GSC description.
- Add the DIs for all 14 GSCs to produce the **total degree of influence (TDI)**.
- Use the TDI in the following equation to compute VAF.
- The final adjusted function point is calculated as,

$$\text{VAF} = (\text{TDI} \times 0.01) + 0.65$$

$$\text{AFP} = \text{UFP} \times \text{VAF}$$

0 = No Influence

1 = Incidental

2 = Moderate

3 = Average

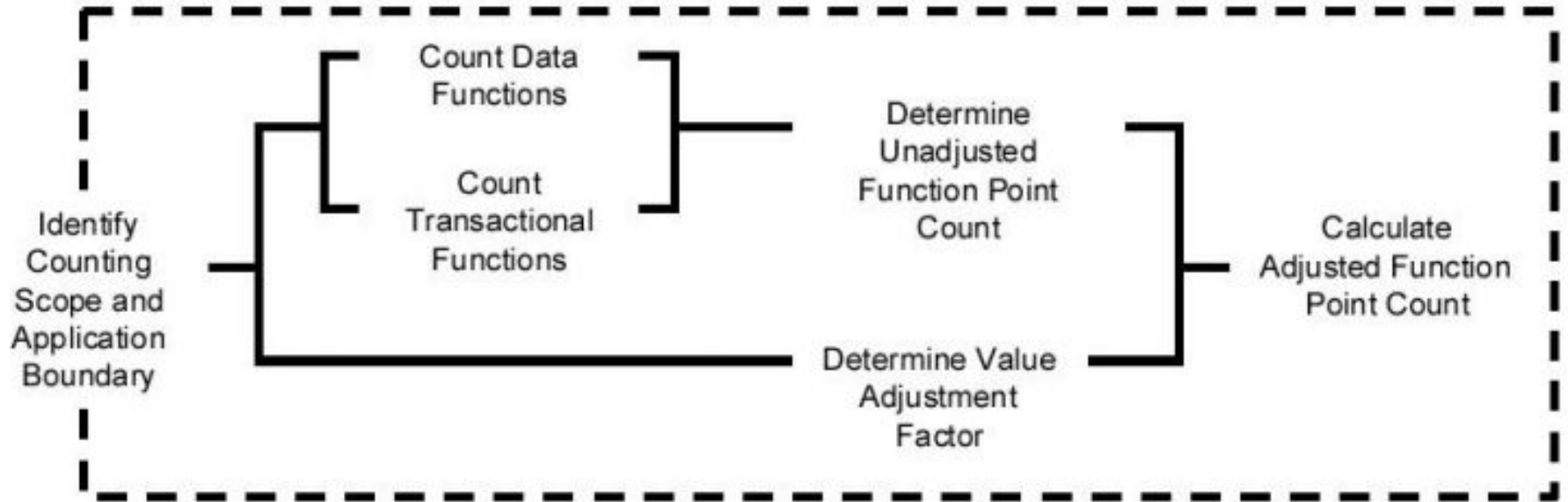
4 = Significant

5 = Essential

Factor	Meaning
F1	Data communications
F2	Performance
F3	Transaction rate
F4	End user efficiency
F5	Complex processing
F6	Installation ease
F7	Multiple sites
F8	Distributed data processing
F9	Heavily used configuration
F10	Online data entry
F11	Online update
F12	Reusability
F13	Operational ease
F14	Facilitate change

Software Metrics

FUNCTION POINT ANALYSIS (FPA)



Software Metrics

FUNCTION POINT ANALYSIS (FPA)

Example:

Consider a project with the following parameters: EI = 50, EO = 40, EQ = 35, ILF = 06, and ELF = 04. Assume all weighing factors are **average**. In addition, the system requires **Significant** performance, **average** end-user efficiency, **moderate** distributed data processing, and **Significant** data communication. Other GSCs are **incidental**. Compute the function points using FPA.

Measurement Parameter	Count		Weighing factor			
			Simple Average Complex			
1. Number of external inputs (EI)	<u>50</u>	*	3	4	6 =	<u>200</u>
2. Number of external Output (EO)	<u>40</u>	*	4	5	7 =	<u>200</u>
3. Number of external Inquiries (EQ)	<u>35</u>	*	3	4	6 =	<u>140</u>
4. Number of internal Files (ILF)	<u>06</u>	*	7	10	15 =	<u>60</u>
5. Number of external interfaces(EIF)	<u>04</u>	*	5	7	10 =	<u>28</u>
UFP Count-total →						628

- 0 = No Influence
- 1 = Incidental
- 2 = Moderate
- 3 = Average
- 4 = Significant
- 5 = Essential

As we know,

$$\begin{aligned}
 \text{Function Point} &= \text{VAF} * \text{UFP} \\
 &= 0.88 * 628 \\
 &= \mathbf{552.64}
 \end{aligned}$$

General System Characteristics (GSCs)	Degree of Influence (DI) 0 - 5
1. Data Communications	<u>4</u>
2. Distributed Data Processing	<u>2</u>
3. Performance	<u>4</u>
4. Heavily Used Configuration	<u>1</u>
5. Transaction Rate	<u>1</u>
6. Online Data Entry	<u>1</u>
7. End-User Efficiency	<u>3</u>
8. Online Update	<u>1</u>
9. Complex Processing	<u>1</u>
10. Reusability	<u>1</u>
11. Installation Ease	<u>1</u>
12. Operational Ease	<u>1</u>
13. Multiple Sites	<u>1</u>
14. Facilitate Change	<u>1</u>
Total Degree of Influence (TDI)	<u>23</u>
Value Adjustment Factor (VAF)	$(23 * 0.01) + 0.65 = 0.88$
	$\text{VAF} = (\text{TDI} * 0.01) + 0.65$

Function Points Vs LOC

Function Point	Line of Code
Function Point metric is specification-based.	LOC metric is based on analogy.
Function Point metric is language independent.	LOC metric is dependent on language.
Function Point metric is user-oriented.	LOC metric is design-oriented.
Function Point metric is extendible to Line of Code.	It is changeable to FP (i.e, backfiring)
Function Point is used for data processing systems	LOC is used for calculating the size of the computer program
Function Point can be used to portray the project time	LOC is used for calculating and comparing the productivity of programmers.